Lahore University of Management Sciences
School of Science and Engineering, Department of Electrical Engineering
AI600 – Deep Learning
Spring 2026

Assignment 3

Date Assigned: 24th March, 2026 **Date Due:** 6th April, 2026

Instructions

- While discussions with your peers is strongly encouraged, please keep in mind that the assignments is to be attempted on an individual level.
- In order to gain the maximum benefit, try to perform solo brainstorming before any consultations. The thought process behind searching for the correct approach all on your own will be invaluable.
- **Late Submissions.** Refer to the Late Flex Days Policy on LMS.
- **Question 2** and **Question 4** are just for practice and **NO** submission is required for them.
- Students must submit their answers, results, and findings in the form of PDF report through the LMS Assignments Tab.
- For coding tasks, students are required to create a public GitHub repository and provide its link in the first page of report as part of the submission.
- Students are welcome to use generative AI tools as partners in their learning journey. However, they are required to explicitly mention the prompt, its output, and edits made in the end of your report. For details, please refer to the AI policy listed on the course outline.

Analytical

This section of the assignment consists of analytical and mathematical questions to be done by hand (no coding). The questions will explore different mathematical concepts and interpretations while working with the same foundational model structure.

Model Architecture Setup: The "Toy-CNN"

The explicit model definition and its corresponding computational graph:

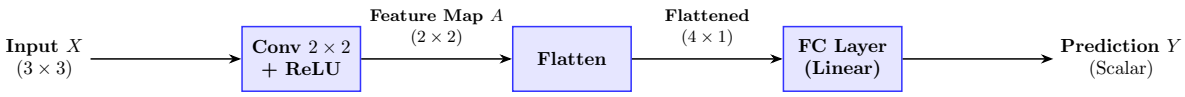

Figure 1: Computational graph of the Toy-CNN. Nodes represent operations, and edges represent the flow of data tensors.

Input: A 3×3 single-channel (grayscale) image, denoted as matrix X.

Layer 1 (Convolutional):

- One 2×2 filter, denoted as matrix W.
- Bias b (a scalar).
- Stride $s = 1$, No padding.
- Activation: ReLU, yielding feature map $A = \text{ReLU}(X * W + b)$.

Layer 2 (Fully Connected):

- The 2×2 feature map A is flattened into a 4×1 vector: $[a_{11}, a_{12}, a_{21}, a_{22}]^T$.
- Weights vector V of size 1×4 and bias c.
- Output prediction: $Y = V \cdot A_{\text{flattened}} + c$.
- Activation: Linear (None).

Loss Function: Mean Squared Error for a single sample: $L = \frac{1}{2}(Y - T)^2$, where T is the true target value.

Initial Weights & Data for the Assignment:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad b = 0$$

$$V = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix}, \quad c = 1$$

$$\text{Target } (T) = 3$$

Q1: Forward and Backward Pass

Objective: Test the mechanical understanding of convolution, ReLU, and basic chain-rule differentiation through a network.

1a) Forward Pass: Compute the pre-activation feature map Z, the post-activation feature map A, the final prediction Y, and the loss L. *(Show all intermediate matrices).*

1b) Backward Pass (Weights): Compute the gradients of the loss with respect to the fully connected weights ($\frac{\partial L}{\partial V}$) and the convolutional filter ($\frac{\partial L}{\partial W}$).

1c) Interpretation: Observe the gradient matrix $\frac{\partial L}{\partial W}$. If we were to perform one step of Gradient Descent, which specific weight in the 2×2 filter W would change the most, and in what direction? Explain structurally why this specific weight has the largest gradient based on the forward pass activations.

Q2: Image Perturbation & Receptive Fields (No submission required)

Objective: Introduce the concept of network sensitivity, adversarial vulnerability, and the spatial effects of weight sharing.

2a) Backward Pass (Input): Compute the gradient of the loss with respect to the center pixel of the input image, $\frac{\partial L}{\partial X_{2,2}}$.

2b) Adversarial Perturbation: Suppose you are an adversary who wants to *maximize* the loss L by modifying only the center pixel $X_{2,2}$ by a tiny amount ϵ. Based on a linear approximation using your gradient from 2a, should ϵ be positive or negative?

2c) Interpretation: The center pixel $X_{2,2}$ is involved in multiple spatial locations in the pre-activation feature map Z. Explain how the architectural concept of "weight sharing" in CNNs causes certain pixels (like the center one) to have a compounding impact on the final gradient compared to a corner pixel like $X_{1,1}$.

Q3: Pooling and Normalization

Objective: Explore how structural changes to the CNN alter the mathematics and the network's behavior.

3a) Max Pooling: Imagine we replace the flattening step and the 4×1 fully connected layer with a 2×2 Global Max Pooling layer applied directly to A. The output Y is now simply the maximum value in A (assume bias $c = 0$). Recompute Y. How does this architectural change alter the flow of gradients back to X compared to Q1? Which pixels in X now have a gradient of zero?

3b) Normalization: Return to the original architecture, but insert a Layer Normalization step immediately after the ReLU activation, applied to the 2×2 matrix A. Calculate the mean μ and variance σ^2 of A, and write out the normalized feature map A_{norm} (assume scale $\gamma = 1$, shift $\beta = 0$, and $\epsilon = 0$ for simplicity).

3c) Interpretation: Max pooling is often said to provide "translation invariance." Based on your calculations in 3a, explain mathematically why shifting the input image X slightly to the left or right might not change the output Y or the resulting loss.

Q4: Residual Networks and Skip Connections (No submission required)

Objective: Understand how skip connections alter the computational graph, influence gradient flow, and aid in model interpretability.

Suppose we modify the architecture of the Toy-CNN to include a residual skip connection around the convolutional layer. Because the input X is 3×3 and the pre-activation feature map Z is 2×2, we will perform a "cropped" identity mapping. We take the top-left 2×2 sub-matrix of X, denoted as X_{crop}, and add it to the output of the ReLU activation.

The new residual feature map is defined as:

$$A_{\text{res}} = A + X_{\text{crop}} = \text{ReLU}(X * W + b) + \begin{bmatrix} X_{1,1} & X_{1,2} \\ X_{2,1} & X_{2,2} \end{bmatrix}$$

The rest of the network (Flattening, Fully Connected Layer, and Loss) remains exactly the same, using A_{res} in place of A.

4a) Forward Pass: Using the original initial weights and data, compute the new residual feature map A_{res} and the new final prediction Y_{res}.

4b) Backward Pass (Gradient Flow): Recompute the gradient of the loss with respect to the top-left pixel of the input image, $\frac{\partial L}{\partial X_{1,1}}$, under this new architecture.

4c) Interpretation: Compare your result in 4b to the gradient flow of the original architecture. Mathematically explain how the term $+X_{\text{crop}}$ acts as a "gradient highway." Why is this structural property crucial for training very deep neural networks without losing the signal from the loss function?

Programming: Implementation and Analysis

In this section, you will transition from manual computations to implementing and training neural networks using PyTorch. You are required to log your training/validation metrics (Loss and Accuracy) and plot them for your report.

Task 1: Custom CNNs and Shortcut Learning

Objective: Design a custom Convolutional Neural Network from scratch, train it on a standard dataset, and investigate how models can exploit spurious correlations rather than learning robust features.

Part A: Standard MNIST

1. **Architecture Design:** Implement a custom CNN in PyTorch to classify the standard MNIST dataset. Your model must adhere to the following constraints:
   - Maximum of 3 Convolutional Layers.
   - Maximum of 2 Fully Connected (Linear) Layers.
   - Total trainable parameters must not exceed 50,000. *(Hint: Use pooling and control your channel dimensions to stay under the limit).*
2. **Training and Evaluation:** Train your model using the Adam optimizer and Cross-Entropy Loss. Plot your training and validation loss, as well as accuracy curves over the epochs. Evaluate the final model on the standard MNIST test set and report the accuracy.
3. **Analytical Question 1.1:** Examine your loss curves. Did your model overfit, underfit, or generalize well? Justify your answer by referencing the gap (or lack thereof) between the training and validation curves.
4. **Analytical Question 1.2:** Extract the weights of the filters from your first convolutional layer. Plot them as images. Can you visually identify any rudimentary edge or texture detection occurring? Briefly explain what these filters seem to be capturing.

Part B: Colored-MNIST (C-MNIST)

Standard MNIST relies on shape. In the real world, models often cheat by relying on background colors or lighting rather than the actual object. We will simulate this using Colored-MNIST dataset provided on LMS.

1. **Adaptation:** Modify your CNN from Part A to accept 3-channel RGB images. Train this new model from scratch on the **C-MNIST Train Set**.
2. **Evaluation:** Evaluate your trained model on two separate test sets:
   - A biased test set.
   - The unbiased test set.
3. **Analytical Question 1.3:** Report the accuracies for both test sets. You should observe a significant drop in performance on the unbiased test set. Explain this phenomenon mathematically and conceptually. Why did the optimizer prefer the color feature over the shape feature during training?
4. **Analytical Question 1.4:** Propose training strategies that could force the network to ignore the color and focus on the shape of the digits (You are not required to provide code for this).

Task 2: Transfer Learning and Interpretability

Objective: Utilize a pre-trained model on a more complex dataset (STL-10) and use GradCAM (Gradient-weighted Class Activation Mapping) to visually interpret the model's decision-making process.

Part A: Fine-tuning ResNet-18

1. **Model Setup:** Load a pre-trained ResNet-18 model provided by `torchvision.models`.
2. **Freezing the Backbone:** Freeze all convolutional layers (the "backbone") of the ResNet-18 model so their gradients are not computed during backpropagation.
3. **Classification Head:** Replace the final fully connected layer of ResNet-18 with a new, randomly initialized linear layer that outputs 10 classes (matching STL-10).
4. **Training:** Train *only* the new classification head on the STL-10 training set. Evaluate on the STL-10 test set and report your accuracy.
5. **Analytical Question 2.1:** Why is it computationally and functionally beneficial to freeze the early layers of a pre-trained network when fine-tuning on a relatively small dataset like STL-10? What type of features are stored in those early layers compared to the later layers?

Part B: Visualizing Decisions with GradCAM

GradCAM allows us to see which regions of the input image were most important for the model's final prediction by utilizing the gradients flowing into the final convolutional layer.

1. **Implementation:** Use an existing library (such as `pytorch-grad-cam`) or implement GradCAM yourself. Extract the activation maps from the final convolutional layer of your fine-tuned ResNet-18 (usually `layer4`).
2. **Visualization:** Select 4 images from the STL-10 test set:
   - 2 images that the model classified **correctly**.
   - 2 images that the model classified **incorrectly**.

   Generate and display the GradCAM heatmaps overlaid on the original images for these 4 samples.
3. **Analytical Question 2.2:** Analyze the heatmaps for the **correct** predictions. Is the model focusing on the primary object (e.g., the dog, the airplane), or is it using background context (e.g., grass, sky) to make its decision?
4. **Analytical Question 2.3:** Analyze the heatmaps for the **incorrect** predictions. Based on the highlighted regions, can you deduce *why* the model made the wrong prediction? (e.g., Did it focus on a confusing background element, or did it mistake a localized feature for another class?)

Report Submission Guidelines

You are required to submit a single PDF report containing all required components of the assignment.

- **Report Preparation:** Students are strongly encouraged to prepare their report using LaTeX. The report should be clearly structured, well-formatted, and easy to follow.
- **Programming Components:** All programming-related parts of the assignment must be published as a public **GitHub repository** with link provided on the **first page** of the report.
- **AI Policy Compliance:** In the end of report, mention your usage of Generative AI tools, clearly stating:
  - The exact prompt(s) used,
  - The corresponding output(s) generated by the tool, and
  - Any edits, or modifications you applied to the generated content.